High-Performance Similarity Search Engine

YZM 513 — Project 1 | Fall 2026

Final Technical Report — 20,000 Qwen-generated documents

Research question. How can a pure-Python semantic similarity search be transformed into a fast, correct, reproducible CPU retrieval pipeline, and how does the resulting system scale from 1,000 to 20,000 documents?

Corpus	20,000 English Qwen2.5-0.5B-Instruct responses
Representations	Sparse TF-IDF (9,617 features) and dense MiniLM (384d)
Final approximate index	TruncatedSVD 100d; 20,000 vectors
Final measured 20K batch	190.792 ms for 100 queries (SVD batched, median of 5 runs)
Exact batched alternative	208.815 ms for 100 queries (384d, median of 5 runs)
Live demonstration	Interactive Colab Top-10 retrieval, tested on real corpus

Measurement warning. Search-only batch timing is not a single-query response time. The live-demo timing is a one-off end-to-end observation. SVD Top-10 overlap is not a human relevance score.

Source repository: github.com/OsmanTurgut2335/HP-SS-Engine | Branch: feature/phase4-final-system

1. Introduction and Experimental Environment

The task is to return the top-k most similar documents to a query, while reducing the computational cost of similarity calculation and ranking. The project proceeds from profiling an intentionally naive implementation to choosing vector representations, dimensionality reduction, vectorized kernels and concurrency strategies. Every claimed improvement is tied to an actual measured workload.

Corpus construction and audit

We generated 20,000 English responses with Qwen2.5-0.5B-Instruct using 80 topics, 10 task styles and 25 contexts, giving 20,000 unique prompt combinations. The JSONL corpus stores IDs, prompts, responses, token counts and model identifiers. A resumable Colab process produced 15K records and later appended 5K.

Statistic	Verified value
Rows and unique IDs	20,000; IDs 0–19,999
Unique prompts	20,000
Mean / median response length	58.13 / 53 model tokens
Repeated response texts	288 additional duplicate occurrences
Near 128-token generation cap	678 responses
Malformed JSON / empty responses	0 / 0

Preprocessing and platform

TF-IDF uses default lowercase word tokenization, L2 normalization and float32 storage; no additional stemming, lemmatization or stopword removal. Dense semantic representations use all-MiniLM-L6-v2, yielding 384 float32 coordinates per document. The final scaling run used Colab Linux, Python 3.13.16, NumPy 2.1.3, two reported CPUs and one BLAS thread.

All performance timings below are CPU-based. Corpus generation used a GPU, but generation time is not included in search benchmarks. The raw corpus and cached embeddings remain in Google Drive; reproducibility notebooks and measured CSV files are linked from GitHub.

2. Pure-Python Baseline and Profiling

Phase 1 implemented cosine similarity using nested Python loops over 512-feature TF-IDF vectors, followed by Top-10 sorting. This deliberate baseline exposes interpreter overhead and repeated arithmetic. Each corpus size used five queries and five timed repetitions (25 observations), with profiling performed separately.

Documents	Median latency per query	Mean latency
100	29.685 ms	30.106 ms
500	125.223 ms	121.429 ms
1,000	228.967 ms	236.177 ms

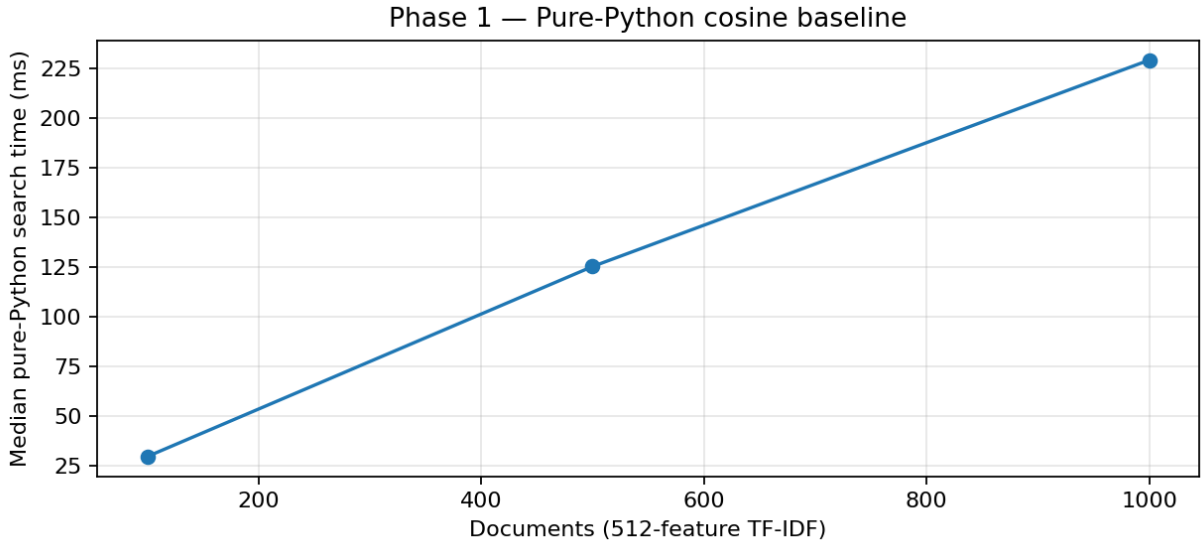


Figure 1. Pure-Python latency rises sharply with corpus size, motivating native vectorized computation.

Measured bottlenecks

At 1K documents, cProfile recorded 1,000 calls to the pure-Python cosine function; its own time was 0.284 s and cumulative time 0.287 s. The line profiler attributed about 99.7% of the inclusive top-k function time to the cosine-call line, versus roughly 0.1% to sorting. Major contributors were Python dot-product loops, repeated norm calculations and loop iteration overhead.

Interpretation: profiling shows where time is spent, not merely that Python is slow. The optimization target is the repeated per-document similarity calculation. Phase 1 uses 512-feature TF-IDF; its timings must not be divided by later 384d MiniLM times as though the workloads were identical.

3. Matrix Operations: Sparse vs. Dense

Phase 2A compared sparse TF-IDF and dense MiniLM embeddings, including custom sparse-sparse, sparse-dense and dense-dense dot-product implementations. Sparse TF-IDF is appropriate for lexical features with many zeros; dense semantic embeddings contain almost no zeros and are more efficient as contiguous arrays.

20K representation	Array storage
TF-IDF CSR (9,617 features)	6.701 MiB
MiniLM dense float32 (384d)	29.297 MiB
MiniLM stored as CSR	58.670 MiB

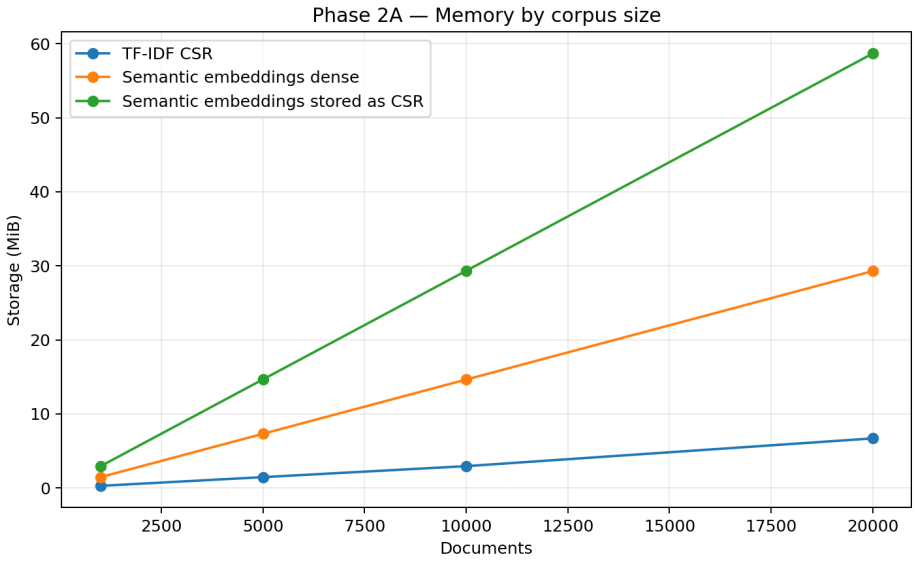


Figure 2. CSR saves memory for sparse TF-IDF, but approximately doubles storage for nearly dense MiniLM embeddings.

Like-for-like timing comparison

At 20K documents, the custom MiniLM sparse-sparse method took 6,066.276 ms median per query, sparse-dense 3,054.274 ms and dense-dense 149.586 ms. These compare the same embedding vectors, although implementations also differ. TF-IDF sparse-sparse took 560.687 ms but is a different representation and should not enter the same speedup ratio.

Design decision: use dense float32 MiniLM embeddings for semantic search; keep TF-IDF CSR as a memory-efficient lexical alternative. This choice reflects both storage structure and measured algorithm cost.

4. Dimensionality Reduction and Design Choice

Phase 2B tested custom block power-iteration SVD, scikit-learn TruncatedSVD, PCA and a full NumPy SVD reference. We measured component counts k=50, 100 and 200 and multiple corpus sizes. The final pipeline retains TruncatedSVD k=100 as a controlled approximate representation; k=100 is a chosen engineering parameter, not an inherent property of MiniLM.

20K, k=100 method	Fit + reconstruction median	Relative Frobenius error
Custom power iteration	5.740 s	0.34428
TruncatedSVD	4.239 s	0.34442
PCA	1.935 s	0.34366

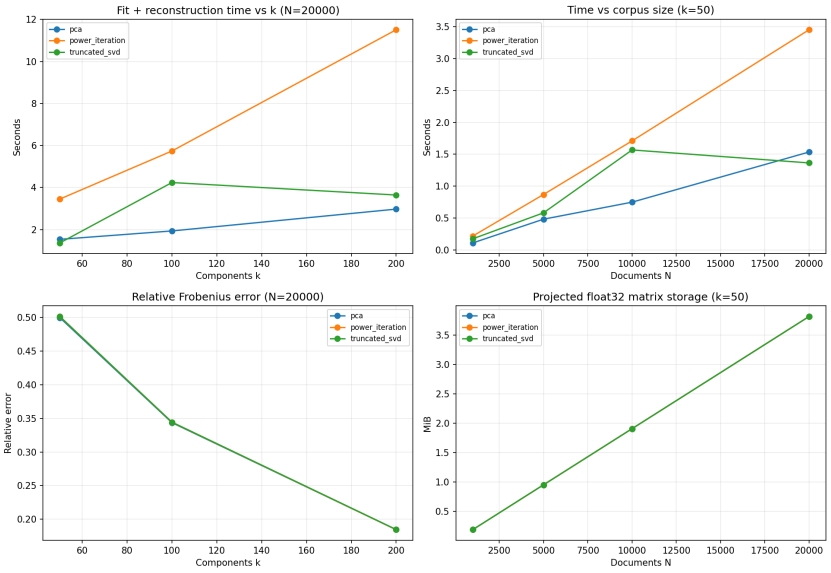


Figure 3. Measured reduction-method cost depends on matrix size, component count and operation definition; TruncatedSVD is not universally the fastest in this run.

Fit-only at 20K, k=100: NumPy full SVD 1.436 s, TruncatedSVD 3.280 s and PCA 3.212 s. This counterexample is reported explicitly rather than claiming randomized SVD always wins. Custom power iteration reached the 35-iteration cap, so convergence was not established.

At 20K, float32 384d storage is 29.297 MiB; 100d is 7.629 MiB (about 74% lower). Phase 2B's five in-corpus query diagnostic gave 94% mean TruncatedSVD Top-10 overlap, but Phase 4B's larger external-query test found 77%. Quality must therefore be reported with the query population.

5. Vectorization vs. Element-wise Loops and JIT

Phase 2C implemented the same Gram-matrix operation in pure Python loops, NumPy matrix multiplication, Numba serial and Numba parallel. The performance comparison isolates the choice of computational kernel while keeping the operation fixed; it does not directly measure Top-10 search.

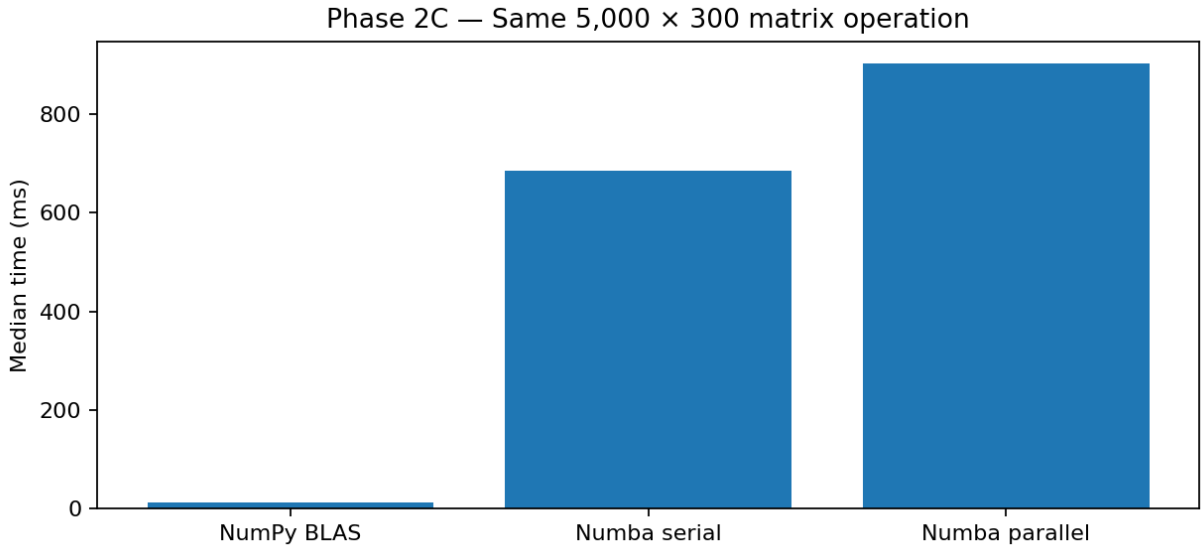


Figure 4. For the 5,000 × 300 matrix workload, optimized NumPy BLAS was much faster than explicit Numba loops.

Method, 5K × 300	Median
NumPy BLAS	11.945 ms
Numba serial	685.929 ms
Numba parallel	902.780 ms

NumPy uses optimized native matrix kernels, efficient contiguous memory access and vectorized operations. Numba eliminates Python interpreter overhead but does not automatically beat tuned BLAS; parallel scheduling, loop structure and memory traffic can overwhelm its benefit.

These three timings are from the Phase 2C Gram benchmark. They are not interchangeable with Phase 3 search timings. The same-vector Phase 3D experiment separately evaluates Numba against BLAS for actual Top-10 retrieval.

6. Parallelism and Concurrency Strategies

Phase 3 tested five strategies across matched groups: batched NumPy/BLAS, Numba prange, ThreadPoolExecutor, ProcessPoolExecutor and asyncio. The objective was to find the best speedup per unit of implementation complexity, not to maximize worker count.

Phase 3B, 20K × 128 queries	Median total time	Speedup vs sequential
Sequential NumPy	670.172 ms	1.00×
Batched BLAS	435.124 ms	1.54×
ThreadPool 4	489.070 ms	1.37×
ProcessPool 4, chunk 8	475.582 ms	1.41×

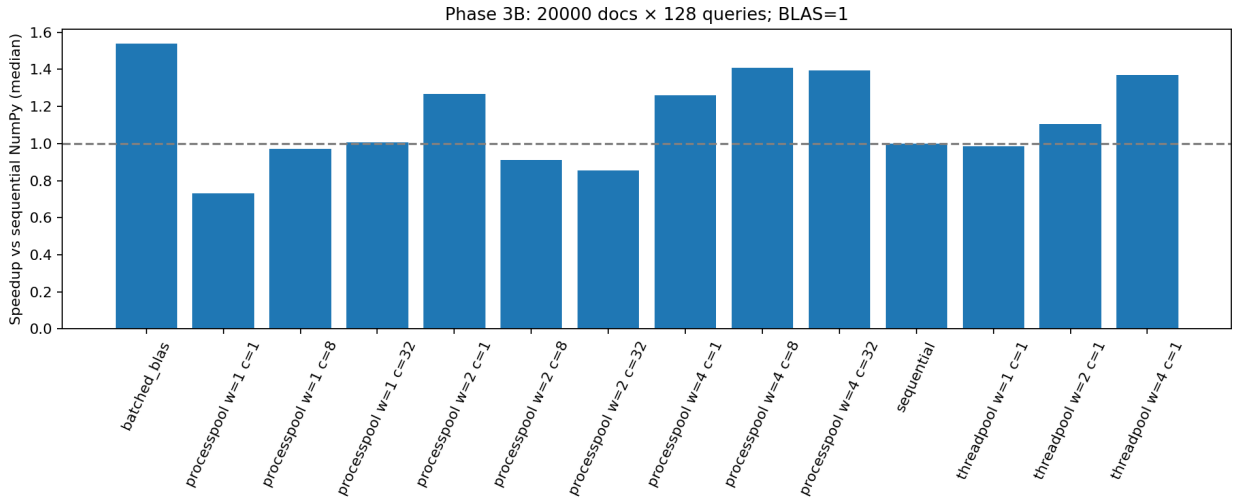


Figure 5. Additional processes do not guarantee a better result: task scheduling and inter-process overhead constrain speedup.

Course-concept interpretation

GIL: Python threads cannot freely parallelize Python bytecode, although NumPy native kernels may release the GIL. **Processes:** independent interpreters bypass the GIL but introduce startup, serialization and IPC costs. **asyncio:** overlaps waiting operations rather than accelerating CPU arithmetic. For 120 simulated 50 ms I/O waits, concurrency 40 reduced elapsed time from 6,040.915 to 154.435 ms; this is an I/O concurrency result, not a CPU search speedup.

Amdahl's Law: serial ranking, orchestration and memory contention limit parallel gains. Cache and memory bandwidth are plausible constraints, but actual bandwidth was not directly profiled.

7. Same-vector Search Benchmark and Fair Comparison

To connect the pure-Python baseline to later optimizations without changing vector representations, Phase 3D repeated Top-10 search on the same normalized 384d MiniLM vectors. Five timed repetitions per configuration were collected; compilation was excluded from timed Numba measurements.

Phase 3D, 20K × 64 queries	Median total time
Sequential NumPy	436.527 ms
Batched BLAS	226.436 ms
Numba serial	942.034 ms
Numba parallel, 2 threads	583.025 ms

In this matched experiment, batching delivered a 1.928× improvement over sequential NumPy. Numba parallel improved over Numba serial by about 1.62× but remained slower than batched BLAS. All 58 recorded Top-10 set and order checks matched; maximum observed score difference was about 1.0133e-6.

Python baseline bridge

At 1K documents × 4 queries, a Phase 1-style pure-Python cosine implementation took 1,687.633 ms versus 0.687793 ms for sequential NumPy, about 2,454× difference. This comparison uses identical MiniLM vectors, but Python recomputes norms while NumPy uses pre-normalized vectors; the ratio captures the complete implementation difference, not interpreter cost alone.

Why batched BLAS was selected

The batched matrix-matrix approach requires relatively little orchestration code, reuses the in-memory document matrix efficiently and outperformed the tested threading/process alternatives in the Phase 3B matched run. It is the most defensible default for dense, in-memory, multi-query retrieval on the measured Colab CPU.

8. Final System Architecture and Design Decisions

The deployed demonstration uses an index prepared offline, not a model retrained at query time. Text responses are embedded once using MiniLM (384d), transformed by the saved TruncatedSVD model (100d), L2-normalized and cached in an array. An incoming query is embedded with the same MiniLM model, transformed by the same SVD, normalized, compared with the cached vectors and ranked.

Pipeline stage	Input → output	Timed during live query?
MiniLM encoding	Text → 384d float32	Yes
SVD projection	384d → 100d	Yes
Cosine BLAS scores	20K × 100d → 20K scores	Yes
Stable Top-10 ranking	Scores → document IDs	Yes
Index loading / SVD fitting	Drive artifacts → memory	No; preloaded

Two defensible deployment modes

Exact mode: keep 384d vectors and use batched BLAS for high-throughput retrieval with reference Top-10 fidelity. **Reduced mode:** use k=100 SVD for lower index memory and somewhat faster measured 100-query batches, while acknowledging approximate-neighbor differences.

Reproducibility

The GitHub branch includes all Phase 4 notebooks, original full-run CSV files, the original Phase 4B and 4C figures, an environment JSON and a detailed findings document. Google Drive retains the full Qwen JSONL corpus, cached MiniLM embeddings, SVD model and reduced index. Colab setup cells install the required packages and mount Drive. The full scaling run uses 100 fixed queries, two warmups, five timed repeats and one BLAS thread.

No GPU is required for search. A model or corpus file missing from Drive must be regenerated using the earlier notebooks before running the final demo; the repository alone does not contain these large artifacts.

9. Phase 4B — Numerical Correctness and Approximation

At 20K documents, 100 in-corpus and 10 external text queries were evaluated. Exact float32 BLAS cosine scores were checked against a float64 reference with a maximum accepted absolute error of 1e-5. Exact Top-10 sets and ordered lists matched for all tested queries.

Query population	Exact Top-10 match	SVD k100 overlap
100 in-corpus queries	100% set and order	86.9% mean
10 external queries	100% set and order	77.0% mean

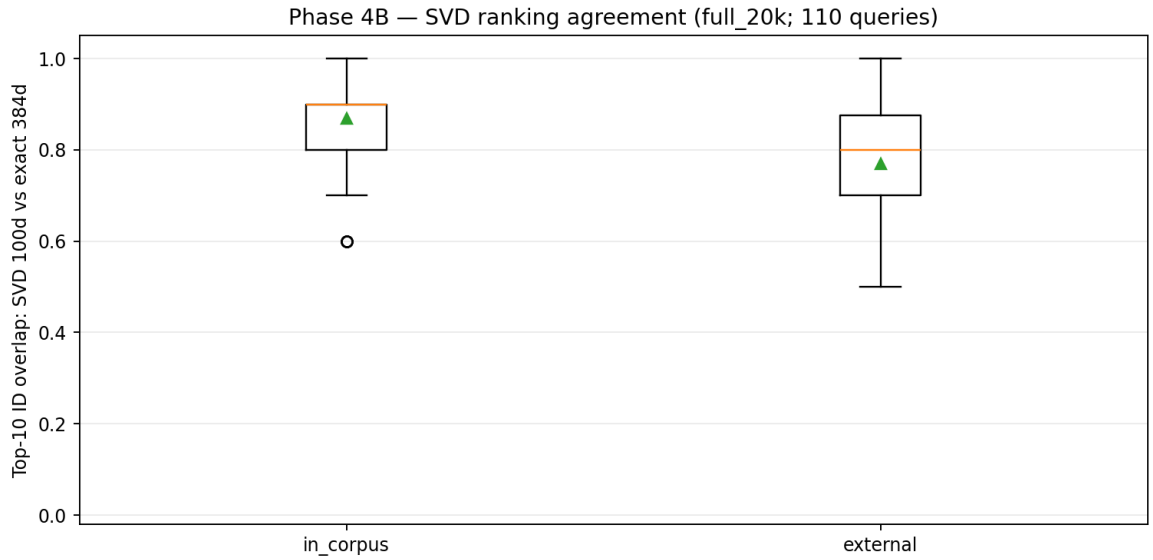


Figure 6. SVD k=100 changes some neighbors; the lower external-query overlap is an important quality tradeoff, not an implementation error.

The maximum absolute score error was 1.46344e-7 for in-corpus and 1.26517e-7 for external queries, comfortably below 1e-5. The discrepancies CSV contains no failures. Exact score agreement and ranking consistency do not establish human-perceived semantic relevance. The in-corpus test can overstate quality due to self-matches.

Quality decision

SVD k100 is acceptable as a measured approximate-search option when memory and throughput matter, but exact 384d retrieval is preferable if preserving the original cosine Top-10 is a hard requirement. We do not claim 77% semantic accuracy; it is overlap with the original representation.

10. Phase 4C — Scaling from 1K to 20K

Using the same 100 fixed embedding queries at each corpus size, Phase 4C measured sequential exact 384d, batched exact 384d and batched SVD 100d. Each configuration had two warmups and five timed repetitions. Reported values are median elapsed times for the full 100-query workload including Top-10 ranking.

Documents	Sequential 384d	Batched 384d	SVD batched 100d
1K	9.479 ms	6.932 ms	8.969 ms
5K	55.020 ms	58.382 ms	47.852 ms
10K	780.963 ms	168.667 ms	148.151 ms
20K	779.859 ms	208.815 ms	190.792 ms

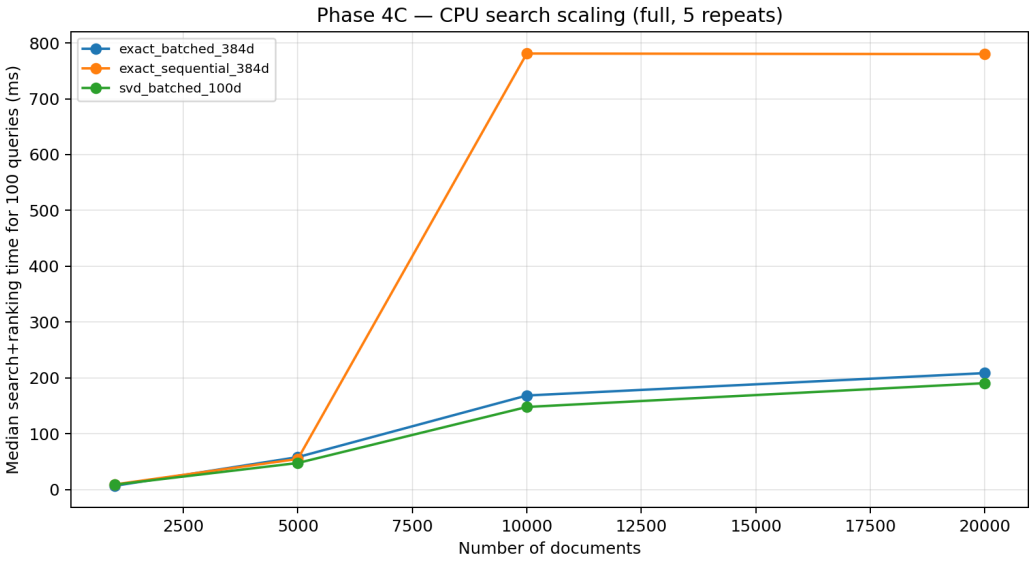


Figure 7. At 20K, batched BLAS yields most of the speedup; timings are not consistently linear and the sequential method shows a marked change near 10K.

At 20K, exact batching improves over sequential by 3.73×; SVD batched improves over sequential by 4.09×, but is only 1.09× faster than exact batched. The latter improvement must be weighed against reduced Top-10 agreement.

These measurements are search-only; MiniLM encoding, SVD fitting, loading and I/O are excluded. A 200K corpus would require approximately 293 MiB for 384d float32 embeddings alone; 200K latency was not measured. Nonlinear timings and run-to-run variance prevent claiming a proven CPU bottleneck without further profiling.

11. Discussion, Lessons and Future Work

What surprised us

• Dense storage was smaller than CSR for nearly full MiniLM embeddings, even though CSR was substantially smaller for sparse TF-IDF.

• Full NumPy SVD fit was faster than TruncatedSVD fit in the measured 20K × 384 configuration; theoretical complexity did not predict the observed runtime by itself.

• Batched BLAS often beat hand-parallelized threads, processes and Numba loops while remaining simpler to maintain.

• At 20K, reducing dimensions added only a modest batch-time improvement beyond exact batching, but decreased raw index memory by roughly 74%.

• The live query's text-encoding step took a substantial share of observed end-to-end latency.

Live demonstration

The interactive Colab widget returned 10 real documents for 'How can AI help detect cyber attacks?'. One observed run took 69.91 ms total: 38.26 ms encoding, 1.25 ms SVD projection and 30.40 ms vector search plus ranking. This is a single observation, not a five-repeat end-to-end benchmark. In the 30-second presentation demo, model and index loading happen before the audience-facing interaction.

What we would do with more time

• Repeat 20K runs on a second CPU and instrument cache misses, memory bandwidth, sorting and BLAS kernel selection.

• Measure 200K or larger corpora instead of extrapolating, and test approximate nearest-neighbor indexes such as HNSW.

• Collect human relevance judgments to distinguish semantic search quality from agreement with original embeddings.

• Benchmark repeated interactive single-query latency separately from batched throughput, including MiniLM inference.

References and artifact index

Course specification: YZM 513 Project 1, High-Performance Similarity Search Engine (Fall 2026). Implementation, complete notebooks, reproducibility instructions and original measured data: github.com/OsmanTurgut2335/HP-SS-Engine/tree/feature/phase4-final-system . Full findings: report/phase4_final_findings.md .

Libraries: NumPy/BLAS, SciPy, scikit-learn, Numba, sentence-transformers, pandas, matplotlib and Python concurrent.futures/asyncio. Source corpus: Qwen2.5-0.5B-Instruct outputs generated for this project. Raw large assets remain in the project's Drive workspace.